

Informatik 2

Kapitel 3 – Grundlagen des Software-Entwurfs

Inhaltsverzeichnis

3.1 Einleitung	2
3.2 Strukturierung des Aufbaus von Software	3
3.3 Strukturierung der Vorgehensweise	4
3.3.1 Top-Down-Ansatz (Schrittweise Verfeinerung)	4
3.3.2 Bottom-Up-Ansatz (Modularisierung)	6
3.4 Integriertes Vorgehensmodell	9

3.1 Einleitung

Nach dem ersten Einstieg in Java-Programme und das Prinzip der Objektorientierung im letzten Kapitel wollen wir nun etwas allgemeiner darauf schauen, nach welchen Prinzipien man strukturiert große und komplexe Programme, die regelmäßig aktualisiert und aktiv gewartet werden müssen, erstellen kann.

In der Praxis begegnen uns häufig solche komplexen Systeme, beispielsweise bei der Programmierung von Produktionsstraßen, der Entwicklung eines Betriebssystems, den mehr als 100 Steuergeräten, die in einem modernen Fahrzeug zusammenspielen, oder bei modernen Text- und Datenverarbeitungsprogrammen. Solche Systeme bestehen häufig aus mehreren 100 000 oder gar Millionen Zeilen Code. Bei einem derartigen Umfang kann man keinen Überblick über die Funktionsweise des Codes bekommen, indem man sich in den Code einliest, und die Erstellung des Codes muss geplant und strukturiert werden. Daher ist eine abstraktere Sichtweise auf ein Projekt nötig, die der **Softwareentwurf** umsetzt.

Merke 3.1: Softwareentwurf

Der Softwareentwurf beschäftigt sich mit der Strukturierung, dem Aufbau und der systematischen Entwicklung von Software.

Der Softwareentwurf ist ein Teilgebiet des Software Engineering (auch: *Softwaretechnik*). Er hat zum Ziel, die Entwicklung und den Aufbau von Software zu systematisierten, zu vereinheitlichen und zu standardisieren. Dies vereinfacht das Zurechtfinden in großen Projekten und erleichtert zudem Erstellung, Pflege, Wartung und Erweiterung von Software. Somit wird sichergestellt, dass die Software langfristig eingesetzt werden kann. Da viele Problemstellungen so groß sind, dass eine einzelne Person nicht das gesamte Softwaresystem dazu entwickeln kann, wird auch ein arbeitsteiliger Prozess angestrebt. Die gesamte Problemstellung muss also in Teilprobleme aufgeteilt werden, die von verschiedenen Personen oder Teams gelöst werden können. Entsprechend gibt es viele verschiedene Berufsbilder im Kontext der Erstellung von Softwaresystemen, unter anderem zum Beispiel das des Software-Architekten.

Beispiel 3.2: Analogie – Architektur und Bau eines Gebäudes

Bei der Planung und dem anschließenden Bau eines Gebäudes wird zunächst ein Plan erstellt, dessen Details nach und nach ausgearbeitet werden, bevor mit der Umsetzung des Plans begonnen wird. Dabei wird systematisch vorgegangen und die Aufgaben werden zwischen verschiedenen Personen aufgeteilt. Das Vorgehen wird durch standardisierte Bauabschnitte und Bauteile erleichtert.

In den folgenden Unterkapiteln gehen wir auf die wesentlichen Aspekte einer Strukturierung des Aufbaus von Software und der Vorgehensweise bei der Erstellung von Software ein.

3.2 Strukturierung des Aufbaus von Software

Zum Aufbau von Software gehören die verwendeten Daten und die möglichen Programmabläufe zur Datenverarbeitung.

Merke 3.3: Aufbau von Software

Der Softwareentwurf beschäftigt sich hinsichtlich des Aufbaus von Software mit der Strukturierung von Daten und Programmabläufen.

In der Vorlesung *Informatik 1* haben wir bereits zwei Arten von Strukturierung kennengelernt: Einerseits die **Strukturierung von Algorithmen**, andererseits die **Strukturierung von Daten**. Für eine strukturierte Darstellung von Algorithmen unter Benutzung verschiedener Kontrollstrukturen (Sequenzen, Verzweigungen, Wiederholungen) haben wir Struktogramme, Programmablaufpläne und Pseudocode besprochen. Diese Darstellungen zeigen den logischen Ablauf eines Algorithmus und helfen beim Verständnis des Vorgehens. Die strukturierte Darstellung von Daten zeigt deren logische Eigenschaften und Beziehungen untereinander. Dies haben wir durch geeignete selbstdefinierte struct-Datentypen mit Datenstrukturinvarianten umgesetzt.

Wie bereits im letzten Kapitel eingeführt benutzen wir in der Vorlesung *Informatik 2* Klassen zur Strukturierung von Daten. Klassen definieren Datentypen mit verschiedenen Komponenten und erweitern struct-Datentypen um zugehörige Verwaltungsmethoden. Im letzten Kapitel haben wir bereits einige Beispiele für in der Java-API vordefinierte Klassen als auch selbstdefinierte Klassen kennengelernt.

Ähnlich wie Funktionen in C zuerst abstrakt als Algorithmus in programmiersprachenunabhängiger Form entworfen werden, bevor man sie implementiert, werden auch Klassen zuerst in programmiersprachenunabhängiger Form modelliert, bevor man sie in in einer objektorientierten Programmiersprache implementiert. Dafür benutzt man **UML (Unified Modeling Language)**. UML dient der programmiersprachenunabhängigen Modellierung, Analyse und Dokumentation komplexer Systeme. Mit ihr können beispielsweise Daten, Funktionalitäten, Abhängigkeiten und Abläufe so präzise beschrieben werden, dass diese anschließend mit einer beliebigen objektorientierten Programmiersprache implementiert werden können. UML vereinigt eine Vielzahl graphischer und textueller Beschreibungsmethoden (Diagrammtypen). Diese lassen sich in zwei Hauptgruppen aufteilen: statische und dynamische Diagramme. Die **statischen Diagramme** (z. B. Klassendiagramm, Objektdiagramm, Paketdiagramm) werden zur Strukturierung von Daten verwendet, **dynamische Diagramme** (z. B. Sequenzdiagramm, Zustandsdiagramm, Aktivitätsdiagramm, Anwendungsfalldiagramm) zur Strukturierung von Programmabläufen.

Merke 3.4: UML

UML dient der programmiersprachenunabhängigen Modellierung, Analyse und Dokumentation komplexer Systeme. UML vereinigt dazu eine Vielzahl statischer und dynamischer graphischer Diagrammtypen.

Da UML zu umfangreich ist, um komplett in der Vorlesung behandelt zu werden, werden wir uns nur mit den Grundkonzepten von Klassendiagrammen und Sequenzdiagrammen beschäftigen und

einige weitere Diagrammtypen anhand von Beispielen kennenlernen. Dadurch entwickeln wir eine Basis für den selbständigen Entwurf von Anwendungen kleiner bis mittlerer Größe und für eine eigenständige weitere Einarbeitung in UML.

3.3 Strukturierung der Vorgehensweise

Bei der Entwicklung eines Programms hat man zunächst zwei Ausgangspunkte:

- Auf der einen Seite benötigt man eine Definition, was das Programm am Ende können soll. Die Beschreibung des Programms geschieht zu Beginn aus der Sicht des Nutzers – es wird beschrieben, **was** der Nutzer mit dem Programm machen möchte. Diese Sicht (des Nutzers) auf das Programm nennt man **deklarativ** (*Was wird getan?*).
- Auf der anderen Seite stellt jede Programmiersprache elementare Operationen, Ablaufstrukturen und Datenstrukturen zur Verfügung, die zur Implementierung der deklarativen Beschreibung des Programms geeignet kombiniert werden müssen.

Man muss also die Frage klären, **wie** die elementaren Operationen zur Implementierung der gewünschten Funktionalität einer Anwendung zusammengesetzt werden können. Diese Sicht (des Softwareentwicklers) auf das Programm nennt man **operationell** (*Wie wird es getan?*). Für die Verbindung der deklarativen mit der operationellen Sicht gibt es verschiedene grundsätzliche Vorgehensweisen. Zwei bewährte Techniken hierfür sind der **Top-Down-Ansatz** und der **Bottom-Up-Ansatz**.

Merke 3.5: Deklarativ versus operationell

Die deklarative Sicht auf ein Programm stellt die Frage *Was leistet das Programm?*. Die operationelle Sicht auf ein Programm stellt hingegen die Frage *Wie ist das Programm implementiert?*.

3.3.1 Top-Down-Ansatz (Schrittweise Verfeinerung)

Beim **Top-Down-Ansatz**, auch schrittweise Verfeinerung genannt, geht man folgendermaßen vor:

1. Zuerst wird (deklarativ) die gewünschte Funktionalität der Anwendung aus Sicht des Benutzers beschrieben. Diese Beschreibung wird programmiersprachenunabhängig formuliert und die verwendeten Operationen und Daten werden nur abstrakt beschrieben.
2. Durch eine **Verfeinerung (Konkretisierung)** der im ersten Schritt benutzten Operationen werden diese in Teiloperationen zerlegt und dabei die zugehörigen (durch die Operationen verarbeiteten) Daten passend konkretisiert. Dadurch erhält man eine erste (operationelle) Umsetzung der gewünschten Funktionalität durch Teilfunktionalitäten, dies jedoch weiterhin nur auf abstrakter Ebene.

3. Schließlich wird der Verfeinerungsschritt solange wiederholt, bis man so zu einer Umsetzung gelangt, die durch die elementaren Operationen, Ablaufstrukturen und Datentypen der gewählten Programmiersprache implementiert werden kann. Dabei wird jeweils an den vorhergehenden Verfeinerungsschritt angeknüpft und die bisherigen Beschreibungen werden so immer weiter konkretisiert.

Beispiel 3.6: Analogie – Architektur und Bau eines Gebäudes

Betrachten wir noch einmal das Beispiel des Gebäudebaus. Bei dessen Planung beginnt man ganz abstrakt mit den Wünschen des Benutzers bezüglich Grundriss, Anzahl der Geschosse, sowie Arten und Größen der Räume. Daraus entsteht ein Bauplan, der dann Schritt für Schritt bis zur Bemusterung verfeinert wird. Hierbei werden immer mehr Details der Umsetzung (des Baus) berücksichtigt, wie die Gebäudetechnik, Statik, Wandstärken, Fussbodenhöhen (und vieles mehr), inklusive einer Feinabstimmung der Arbeitsabläufe. Am Ende ist klar, was wann und wie gebaut werden soll.

Aus *Informatik 1* kennen wir bereits eine mögliche Umsetzung des Top-Down-Ansatzes. Zunächst haben wir ein Problem formuliert und aus Sicht des Benutzers in Form einer Problemspezifikation beschrieben (formalisiert). Anschließend kam eine (noch) programmiersprachenunabhängige Umsetzung durch einen Algorithmus. Im letzten Schritt wurde dieser Entwurf dann in eine Programmierung des Algorithmus in C übersetzt.

In *Informatik 2* werden wir etwas größere Systeme entwerfen und implementieren. Ein grobes Vorgehensmodell für den Software-Entwurf solcher Systeme wurde schon kurz in Kapitel 1 vorgestellt und wird als Top-Down-Vorgehen in 3 Phasen in Abbildung 1 gezeigt.

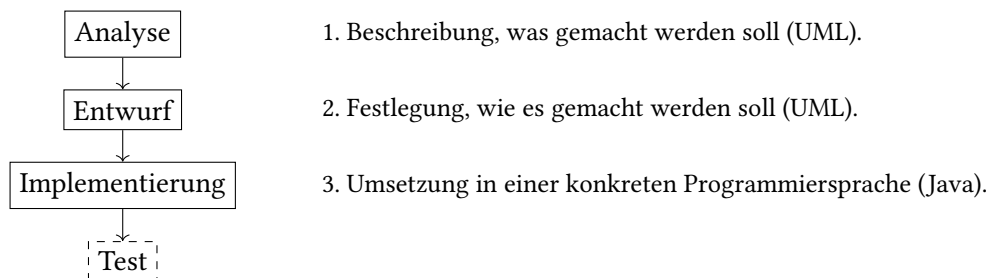


Abbildung 1: Erstellung von komplexen Softwaresystemen als Top-Down-Vorgehen in 3 Phasen.

Für die Analysephase gibt es verschiedene mögliche Ausgangspunkte:

- *Beschreibung der Systemidee* in einigen Sätzen. Dabei wird erläutert, was mit dem System erreicht werden soll, aber nicht, wie dieses Ziel erreicht wird.
- Bei einer *Dokumentenanalyse* werden gewünschte Eigenschaften und Funktionalitäten des Systems vorhandenen Dokumenten entnommen, wie zum Beispiel Berichten, Datenformularen und Bedienungsanleitungen.

- Wenn ein vorhandenes System veraltet ist und neu implementiert werden soll, können das bestehende System (Bildschirmmasken, Benutzerdialoge, und so weiter) und dazu vorhandene Anleitungen als Systembeschreibung verwendet werden. Das nennt man *Reengineering*. Dabei muss jedoch immer überprüft werden, was bei der Neuimplementierung verbessert werden kann und ob möglicherweise Teile des Systems nicht mehr gebraucht werden oder zusätzliche Funktionalität ergänzt werden soll.
- Eine weitere Variante zur Beschreibung des gewünschten Systems ist das sogenannte *Pflichtenheft*. Darin wird eine detaillierte textuelle Beschreibung dessen verfasst, was das System leisten soll.

Manche UML-Diagrammarten lassen sich eindeutig einer bestimmten Modellierungsphase zuordnen, in der sie verwendet werden, andere Diagrammarten werden sowohl in der Analyse-, als auch in der Entwurfsphase genutzt (dabei werden in der Analysephase jedoch nicht alle Aspekte des Diagramms berücksichtigt). Der Übergang zwischen Analyse und Entwurf ist in der Praxis oft fließend.

Die Durchführung der Analysephase und die Trennung von Analyse- und Entwurfsphase können am besten in größeren praktischen Projekten nachvollzogen werden. Beides kann deshalb nicht sinnvoll im Rahmen der Vorlesung *Informatik 2* behandelt werden (Einblicke hierzu erhalten Sie in den späteren Veranstaltungen *Softwareprojekt* und *Softwaretechnik*). In *Informatik 2* werden wir uns also nicht näher mit der Analysephase beschäftigen, sondern immer direkt mit der Entwurfsphase beginnen. Als Ausgangspunkt für die Erstellung eines Systems werden wir uns dabei oft auf eine kurze Beschreibung der Systemidee beschränken.

3.3.2 Bottom-Up-Ansatz (Modularisierung)

Beim Top-Down-Ansatz haben wir uns ausgehend von einer deklarativen Beschreibung eines Programms Schritt für Schritt einer konkreten Umsetzung genähert. Umgekehrt geht man beim **Bottom-Up-Ansatz**, auch Modularisierung genannt, von bereits bestehenden Komponenten (d. h. gelösten Teilproblemen) aus. Zunächst wird die gewünschte Funktionalität in abgeschlossene, allgemein verwendbare Teilfunktionalitäten (Teilprobleme, Teilaufgaben) zerlegt, die den Einsatz vorgefertigter Lösungen erlauben. Anschließend werden diese Teilaufgaben durch allgemein einsetzbare Programmfragmente, sogenannte **Module**, umgesetzt. Im letzten Schritt werden diese Module zu größeren Einheiten zusammengefügt, die die vorgegebene (komplette) Funktionalität umsetzen.

Beispiel 3.7: Analogie – Architektur und Bau eines Gebäudes

Auch beim Gebäudebau findet Modularisierung statt. So werden zum Beispiel Fenster, Türen, Böden, Sanitärinstallation, Elektroinstallation, usw. unabhängig voneinander angefertigt bzw. durchgeführt.

Damit die Teilaufgaben der Zerlegung getrennt voneinander bearbeitet und umgesetzt werden und die resultierenden Module in verschiedenen Projekten (unterschiedlich) kombiniert eingesetzt werden können, sollten sie einige Eigenschaften erfüllen. Jedes Modul muss klar von ande-

ren Modulen abgegrenzt und in sich abgeschlossen sein. Nur so kann es unabhängig von anderen Modulen entwickelt werden. Zudem müssen so später beim Zusammensetzen der Module zu einer Gesamtlösung keine Abhängigkeiten zwischen den verschiedenen Modulen berücksichtigt werden. Ein weiterer Effekt ist die Möglichkeit, ein Modul später durch ein alternatives Modul, welches dieselbe Teilaufgabe erledigt, aber auf andere Art umgesetzt ist, austauschen zu können, ohne dass Seiteneffekte entstehen (das heißt ohne dass auch andere Module geändert werden müssen).

Der Bottom-Up-Ansatz bietet einige Vorteile. Beispielsweise wird die Komplexität eines Problems durch die Zerlegung in überschaubare Teilprobleme reduziert. Außerdem können die einzelnen Module unabhängig voneinander erstellt und getestet werden. Dies ermöglicht eine einfache Aufteilung der Arbeit auf mehrere Personen oder Teams. Darüber hinaus sind die einzelnen Module austauschbar und erweiterbar. Dadurch kann oftmals ein einfacher Übergang zu einer Nachfolgeversion der Software ermöglicht werden.

Merke 3.8: Module (allgemein)

Ein Modul ist eine Softwarekomponente mit folgenden Eigenschaften:

- Es hat eine allgemein einsetzbare und von anderen Modulen abgegrenzte Funktionalität.
- Es kann unabhängig von anderen Modulen entwickelt werden.
- Seine Implementierung ist unabhängig von der Implementierung anderer Module.
- Es kann mit anderen Modulen zu einem größeren System kombiniert werden.
- Seine Implementierung kann jederzeit ohne Seiteneffekte auf andere Module ausgetauscht werden.

Diese Eigenschaften stellt man im Wesentlichen dadurch sicher, dass bei der Kombination von Modulen deren interne Umsetzungen nicht benutzt werden. Vielmehr erfolgt die Kombination über vorab definierte, deklarativ spezifizierte **Schnittstellen** zur Umgebung. Die Schnittstelle eines Moduls verbirgt dessen Umsetzungsdetails. Dadurch findet eine Trennung zwischen der deklarativen Sicht auf ein Modul (*Was wird getan?* = Schnittstelle des Moduls) und der operationellen Sicht (*Wie wird es getan?* = interne Umsetzung des Moduls) statt.

Beispiel 3.9: Analogie – Architektur und Bau eines Gebäudes

Der Anschlüsse verschiedener Module wie Heizung, Sanitärobjekten, Fenster und Türen können unabhängig voneinander erstellt werden. Außerdem kann beispielsweise eine Tür durch eine beliebige andere Tür mit gleicher Schnittstelle (Größe, Position und Art der Scharniere), aber anderer Umsetzung (Material), ausgetauscht werden.

In *Informatik 1* haben wir bereits Module zur Lösung von Teilaufgaben in Form von Header-Dateien eingesetzt. Beispielsweise wurde für die Daten-Ein- und -Ausgabe das Modul `stdio.h` verwendet und für die Verwaltung von Zeichenketten das Modul `string.h`. Die Schnittstelle dieser Module

besteht jeweils aus den zur Verfügung gestellten Prototypen der Verwaltungsfunktionen für die Umsetzung bestimmter Teilfunktionalitäten in einer größeren Anwendung. Zudem haben wir in *Informatik 1* auch eigene Module entwickelt, zum Beispiel für dynamische Datenstrukturen, die wir durch struct-Datentypen und die zugehörigen Verwaltungsfunktionen umgesetzt haben. Dabei war die konkrete Implementierung der Verwaltungsfunktionen nicht relevant für deren Benutzung. Solange der Prototyp gleich bleibt, kann die jeweilige Implementierung daher ausgetauscht und die Funktion trotzdem genau wie zuvor verwendet werden. Bei der Funktion zur Berechnung der Länge einer einfach verketteten Liste ist es beispielsweise für den Benutzer nicht relevant, ob die Funktion rekursiv oder iterativ implementiert wurde.

Mit der Verwendung der Funktionsprototypen als Schnittstelle erreichten wir in *Informatik 1* noch einen weiteren Zweck: den **Schutz von Datenstrukturinvarianten**. Zum Beispiel enthält eine Arraylist ein Array zum Speichern der Elemente und eine ganzzahlige Variable zum Speichern der Anzahl der Elemente. In einer Datenstrukturinvariante haben wir festgelegt, dass der Wert der Variable für die Anzahl der Elemente immer der Länge des Arrays der gespeicherten Elemente entsprechen soll. Für den Zugriff auf eine Arraylist haben wir außerdem verschiedene Verwaltungsfunktionen deklariert (beispielsweise zum Einfügen und Löschen von Elementen) und diese so implementiert, dass bei deren Benutzung diese Datenstrukturinvariante eingehalten wird. Deshalb haben wir in *Informatik 1* folgenden Grundsatz für die Erstellung von C-Programmen aufgestellt:

Der Zugriff auf die Variablen eines struct-Datentyps soll ausschließlich über die zur Verfügung gestellten Verwaltungsfunktionen stattfinden.

Verändert der Benutzer beispielsweise den Wert der Variable für die Anzahl der Elemente einer Arraylist direkt, ohne Elemente einzufügen oder zu löschen, wird obige Datenstrukturinvariante verletzt und nachfolgende Aufrufe von Verwaltungsfunktionen führen zu unerwarteten und nicht korrekten Ergebnissen.

Um solche Fehler zu vermeiden, hat *David Parnas* folgendes Prinzip des **Information Hiding** (**Datenkapselung, Einkapselung**) vorgeschlagen:

Definition 3.10: Prinzip der Datenkapselung nach David Parnas / Information Hiding

Bestimmte Programmteile (Daten, Funktionen), die inhaltlich zusammengehören, werden zu einer Einheit zusammengefasst. Eine solche Einheit soll eine bestimmte Funktionalität bereitstellen, wobei der Anwender der Einheit die Implementierung nicht kennt. Eine solche Einheit soll vor direktem Zugriff geschützt werden. Die Benutzung darf nur über eine klar definierte Schnittstelle erfolgen, eine direkte Verwendung der Implementierung ist nicht möglich. Andernfalls wäre die Austauschbarkeit in einem größeren Programm nicht mehr gewährleistet, da das Gesamtprogramm dadurch möglicherweise nicht mehr wie erwartet funktioniert. Durch diese Kapselung ist auch die Sicherstellung von Datenstrukturinvarianten möglich.

Offenbar erfüllen in Headerdateien deklarierte struct-Datentypen in C dieses Prinzip **nicht**, denn ein direkter Zugriff auf Datenkomponenten kann syntaktisch nicht verhindert werden.

Wir wollen nun den bereits informal eingeführten Begriff eines Moduls genauer definieren. Als zentrale Eigenschaft werden Module hierbei das Prinzip der Datenkapselung erfüllen.

Definition 3.11: Modul (technisch)

Ein **Modul** besteht aus einer Sammlung von Variablenvereinbarungen (Komponenten einer Datenstruktur) mit zugehörigen Datenstrukturinvarianten, und aus Methoden (zugehörige Verwaltungsfunktionen). Variablen und Funktionen werden aufgeteilt in **öffentlich** und **privat**. Die öffentlichen Variablen und Methoden sind außerhalb des Moduls verfügbar und bilden somit die **Schnittstelle** (des Moduls). Die privaten Variablen und Methoden sind nur innerhalb des Moduls verfügbar und sind damit **eingekapselt**, sodass kein Zugriff von außerhalb möglich ist. Alle Methoden werden so implementiert, dass die vorgesehenen Datenstrukturinvarianten erhalten bleiben.

In UML und Java werden solche Module genau durch die bereits in Kapitel 2 eingeführten **Klassen** umgesetzt.

3.4 Integriertes Vorgehensmodell

In der Praxis werden die beschriebenen Vorgehensweisen (*Top-down* und *Bottom-up*) geeignet kombiniert und die beschriebenen Elemente zur Strukturierung des Aufbaus von Software in ein kombiniertes Vorgehensmodell integriert. Abbildung 2 zeigt ein vereinfachtes solches integriertes Vorgehensmodell, an dem wir uns in *Informatik 2* orientieren wollen.

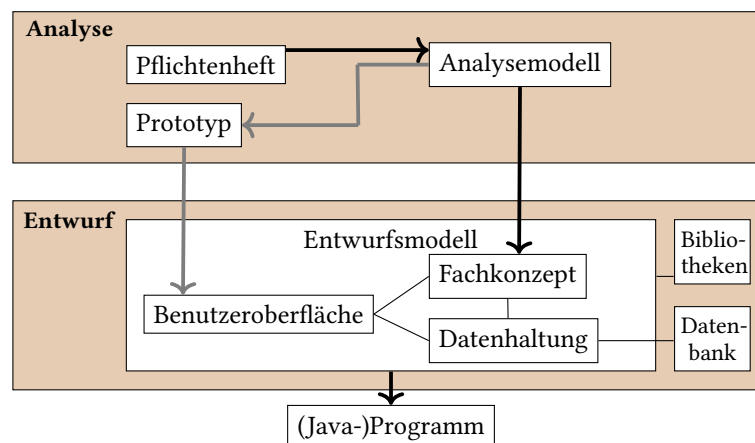


Abbildung 2: Erstellung von komplexen Softwaresystemen als integriertes Vorgehensmodell.

In der Analysephase wird basierend auf dem **Pflichtenheft** (und anderen Quellen) ein **Analysemodell** erstellt. Darin arbeitet man die Anforderungen an Daten und Datenverarbeitung heraus und schätzt ab, ob man diese tatsächlich realisieren kann. Basierend auf den Anforderungen erstellt man dann einen **Prototypen** der Benutzeroberfläche (der zum Beispiel einfach aus analogen oder digitalen Zeichnungen bestehen kann). Mit diesem kann man dem Auftraggeber zeigen, wie

das Programm für den Benutzer aussehen könnte und so sicherstellen, dass er das bekommt, was er haben möchte.

Vom Prototyp ausgehend kann man in der Entwurfsphase eine konkrete **Benutzeroberfläche** ausarbeiten. Aus dem **Analysemodell** entsteht das sogenannte **Fachkonzept**. Dieses enthält eine Beschreibung der Daten und der Datenverarbeitung zur Implementierung der Anforderungen. In der **Datenhaltung** wird das Laden und Speichern von Daten modelliert, wobei es bei einigen Anwendungen sinnvoll ist, eine **Datenbank** anzubinden. Die Benutzeroberfläche stellt die Daten des Fachkonzepts übersichtlich dar und ist das, was der Benutzer schließlich zu sehen bekommt. Falls nötig, werden externe **Bibliotheken** zur Umsetzung des Systems mit eingebunden. Das Entwurfsmodell besteht aus Fachkonzept, Datenhaltung und Benutzeroberfläche.

Im letzten Schritt wird das **Entwurfsmodell** als (Java-)Programm implementiert.

Bevor Sie mit dem nächsten Kapitel beginnen, nehmen Sie sich etwas Zeit und versuchen Sie, die verschiedenen beschriebenen Elemente zur Strukturierung des Aufbaus von Software und der Vorgehensweisen bei deren Erstellung im Modell in obiger Abbildung wiederzufinden und zuzuordnen.